

Start coding or [generate](#) with AI.

```
import pandas as pd
import re

# Load the dataset
df = pd.read_csv('/content/Twitter_Data.csv')

# Display the first few rows and column names to ensure correct loading and ide
print("Original DataFrame head:")
print(df.head())
print("\nDataFrame columns:")
print(df.columns)

# Define the preprocessing function
def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    # Remove user mentions
    text = re.sub(r'@\w+', '', text)
    # Convert to lowercase
    text = text.lower()
    # Remove special characters, punctuation, and numbers, keeping only alphabe
    text = re.sub(r'^a-zA-Z\s]', '', text)
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Apply the preprocessing function to the 'clean_text' column
df['cleaned_text'] = df['clean_text'].apply(preprocess_text)

# Display the DataFrame head with the new 'cleaned_text' column
print("\nDataFrame head after preprocessing:")
print(df[['clean_text', 'cleaned_text']].head())
```

Original DataFrame head:

	clean_text	category
0	when modi promised “minimum government maximum...	-1.0
1	talk all the nonsense and continue all the dra...	0.0
2	what did just say vote for modi welcome bjp t...	1.0
3	asking his supporters prefix chowkidar their n...	1.0
4	answer who among these the most powerful world...	1.0

DataFrame columns:

```
Index(['clean_text', 'category'], dtype='object')
```

DataFrame head after preprocessing:

```
clean_text \
```

```

0 when modi promised "minimum government maximum...
1 talk all the nonsense and continue all the dra...
2 what did just say vote for modi welcome bjp t...
3 asking his supporters prefix chowkidar their n...
4 answer who among these the most powerful world...

```

cleaned_text

```

0 when modi promised minimum government maximum ...
1 talk all the nonsense and continue all the dra...
2 what did just say vote for modi welcome bjp to...
3 asking his supporters prefix chowkidar their n...
4 answer who among these the most powerful world...

```

```

import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
import numpy as np

# Download necessary NLTK data (if not already downloaded)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab') # Added this line to address the LookupError
nltk.download('averaged_perceptron_tagger_eng') # Added this line to address th

# Define a function for POS tagging
def get_pos_tags(text):
    if not isinstance(text, str) or not text.strip():
        return [] # Return an empty list for non-string or empty texts
    tokens = word_tokenize(text)
    pos_tags = pos_tag(tokens)
    return pos_tags

# Handle potential NaN values before applying the function
df['cleaned_text'] = df['cleaned_text'].replace(np.nan, '')

# Apply the POS tagging function to the 'cleaned_text' column
df['pos_tags'] = df['cleaned_text'].apply(get_pos_tags)

# Display the first few rows with 'cleaned_text' and 'pos_tags'
print("\nDataFrame head with POS tags:")
print(df[['cleaned_text', 'pos_tags']].head())

```

```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-

```

```
[nltk_data]      date!
```

DataFrame head with POS tags:

```

                                cleaned_text \
0  when modi promised minimum government maximum ...
1  talk all the nonsense and continue all the dra...
2  what did just say vote for modi welcome bjp to...
3  asking his supporters prefix chowkidar their n...
4  answer who among these the most powerful world...

                                pos_tags
0  [(when, WRB), (modi, NN), (promised, VBD), (mi...
1  [(talk, NN), (all, PDT), (the, DT), (nonsense,...
2  [(what, WP), (did, VBD), (just, RB), (say, VB)...
3  [(asking, VBG), (his, PRP$), (supporters, NNS)...
4  [(answer, NN), (who, WP), (among, IN), (these,...

```

```

from collections import defaultdict
import pandas as pd

# 1. Initialize defaultdict objects for counts
initial_tag_counts = defaultdict(int)
transition_counts = defaultdict(int)
emission_counts = defaultdict(int)
tag_frequency = defaultdict(int)
total_sentences_with_tags = 0

# 2. Iterate through each list of POS tags to populate counts
for pos_tags_list in df['pos_tags']:
    if pos_tags_list: # Check if the list of POS tags is not empty
        total_sentences_with_tags += 1

        # Increment initial tag count
        initial_tag_counts[pos_tags_list[0][1]] += 1

        for i in range(len(pos_tags_list)):
            current_word = pos_tags_list[i][0]
            current_tag = pos_tags_list[i][1]

            # Increment overall tag frequency
            tag_frequency[current_tag] += 1

            # Increment emission counts (tag, word)
            emission_counts[(current_tag, current_word)] += 1

            # Increment transition counts (prev_tag, current_tag)
            if i < len(pos_tags_list) - 1:
                next_tag = pos_tags_list[i+1][1]
                transition_counts[(current_tag, next_tag)] += 1

# 3. Calculate HMM probabilities

```

```

# Initial Probabilities
initial_probs = {
    tag: count / total_sentences_with_tags
    for tag, count in initial_tag_counts.items()
}

# Transition Probabilities
transition_probs = {}
for (prev_tag, current_tag), count in transition_counts.items():
    if tag_frequency[prev_tag] > 0:
        transition_probs[(prev_tag, current_tag)] = count / tag_frequency[prev_
    else:
        transition_probs[(prev_tag, current_tag)] = 0.0 # Should not happen if

# Emission Probabilities
emission_probs = {}
for (tag, word), count in emission_counts.items():
    if tag_frequency[tag] > 0:
        emission_probs[(tag, word)] = count / tag_frequency[tag]
    else:
        emission_probs[(tag, word)] = 0.0 # Should not happen if data is consis

# 4. Convert probabilities to pandas DataFrames for display

# Initial Probabilities DataFrame
df_initial_probs = pd.DataFrame(list(initial_probs.items()), columns=['Tag', 'P
print("\n--- Initial Probabilities (Top 10) ---")
print(df_initial_probs.head(10))

# Transition Probabilities DataFrame
df_transition_probs = pd.DataFrame([
    {'Prev_Tag': prev, 'Current_Tag': curr, 'Probability': prob}
    for (prev, curr), prob in transition_probs.items()
])
# Pivot for better readability, fill NaN with 0 for non-existent transitions
df_transition_probs_pivot = df_transition_probs.pivot_table(
    index='Prev_Tag', columns='Current_Tag', values='Probability', fill_value=0
)
print("\n--- Transition Probabilities (Snapshot) ---")
print(df_transition_probs_pivot.head(5).iloc[:, :5]) # Display first 5x5 for sr

# Emission Probabilities DataFrame
df_emission_probs = pd.DataFrame([
    {'Tag': tag, 'Word': word, 'Probability': prob}
    for (tag, word), prob in emission_probs.items()
])
# Pivot for better readability, fill NaN with 0 for non-existent emissions
df_emission_probs_pivot = df_emission_probs.pivot_table(
    index='Tag', columns='Word', values='Probability', fill_value=0
)

```

```
print("\n--- Emission Probabilities (Snapshot) ---")
print(df_emission_probs_pivot.head(5).iloc[:, :5]) # Display first 5x5 for snap
```

```
--- Initial Probabilities (Top 10) ---
```

	Tag	Probability
0	WRB	0.031276
1	NN	0.336512
2	WP	0.017654
3	VBG	0.020114
4	JJ	0.128920
5	DT	0.074345
6	IN	0.034319
7	NNS	0.071695
8	CD	0.007142
9	VBD	0.012984

```
--- Transition Probabilities (Snapshot) ---
```

Current_Tag	\$	' '	CC	CD	DT
Prev_Tag					
\$	0.000000	0.000000	0.000000	0.000000	0.000000
' '	0.000000	0.000000	0.000000	0.000000	0.176471
CC	0.000011	0.000000	0.002426	0.004239	0.062329
CD	0.000000	0.000139	0.013390	0.008811	0.058069
DT	0.000000	0.000000	0.005174	0.007480	0.013142

```
--- Emission Probabilities (Snapshot) ---
```

Word	a	aa	aaa	aaaa	aaaaagg
Tag					
\$	0.000000	0.0	0.0	0.0	0.0
' '	0.000000	0.0	0.0	0.0	0.0
CC	0.000000	0.0	0.0	0.0	0.0
CD	0.000000	0.0	0.0	0.0	0.0
DT	0.000789	0.0	0.0	0.0	0.0

```
import numpy as np
```

```
# Select a representative tweet by index
```

```
tweet_index = 12345 # Example index, choose one that is relatively short and cl
```

```
selected_tweet_text = df.loc[tweet_index, 'cleaned_text']
```

```
selected_tweet_pos_tags = df.loc[tweet_index, 'pos_tags']
```

```
print(f"--- Selected Tweet ---")
```

```
print(f"Original Index: {tweet_index}")
```

```
print(f"Cleaned Text: {selected_tweet_text}")
```

```
print(f"Original POS Tags: {selected_tweet_pos_tags}")
```

```
# Explain the choice of tweet
```

```
print("\nRationale for tweet selection: This tweet is relatively short and cont
```

ling', 'VBG'), ('exploiting', 'VBG'), ('all', 'PDT'), ('the', 'DT'), ('mechanisms
hout being overly complex.

```
import math

# Get the words from the selected tweet
words = selected_tweet_text.split()
first_word = words[0]

# Get all possible tags from the initial probabilities
all_tags = list(initial_probs.keys())

# Initialize Viterbi table and backpointer table for the first word
viterbi_table = [{}]
backpointer_table = [{}]

print(f"\n--- Viterbi Initialization for word: '{first_word}' ---")

# Step 1: Initialization for the first word
for tag in all_tags:
    initial_prob = initial_probs.get(tag, 1e-10) # Use a small non-zero probabi
    emission_prob = emission_probs.get((tag, first_word), 1e-10) # Use a small

    # Calculate probability: P(tag | start) * P(word1 | tag)
    # Using log probabilities to avoid underflow
    prob = math.log(initial_prob) + math.log(emission_prob)

    viterbi_table[0][tag] = prob
    backpointer_table[0][tag] = None # No backpointer for the first word

# Sort and display top initial probabilities for readability
sorted_initial_probs = sorted(viterbi_table[0].items(), key=lambda item: item[1])

print("Top 5 initial probabilities for each tag emitting the first word:")
for tag, prob_log in sorted_initial_probs[:5]:
    print(f" Tag: {tag}, Log Probability: {prob_log:.4f}, Actual Probability:

# Store the results for the next step
# print("\nViterbi Table after Initialization:", viterbi_table)
# print("Backpointer Table after Initialization:", backpointer_table)
```

```
--- Viterbi Initialization for word: 'fact' ---
Top 5 initial probabilities for each tag emitting the first word:
Tag: NN, Log Probability: -7.8228, Actual Probability: 0.00040051
Tag: JJ, Log Probability: -25.0744, Actual Probability: 0.00000000
Tag: RB, Log Probability: -25.3341, Actual Probability: 0.00000000
Tag: DT, Log Probability: -25.6249, Actual Probability: 0.00000000
Tag: NNS, Log Probability: -25.6612, Actual Probability: 0.00000000
```

```

print(f"\n--- Viterbi Recursion (Forward Pass) for

# Step 2: Recursion (Forward Pass) for subsequent w
for i in range(1, len(words)):
    current_word = words[i]
    viterbi_table.append({})
    backpointer_table.append({})

    print(f"\nProcessing word {i+1}: '{current_word

    for current_tag in all_tags:
        max_prev_prob = -math.inf
        best_prev_tag = None

        emission_p = emission_probs.get((current_ta
        log_emission_p = math.log(emission_p)

        for prev_tag, prev_log_prob in viterbi_tabl
            transition_p = transition_probs.get((pr
            log_transition_p = math.log(transition_

            # P(path to prev_tag) * P(current_tag |
            # In log space: log(P(path to prev_tag)
            current_prob = prev_log_prob + log_tran

            if current_prob > max_prev_prob:
                max_prev_prob = current_prob
                best_prev_tag = prev_tag

        # Store the max probability and the backpoi
        viterbi_table[i][current_tag] = max_prev_pr
        backpointer_table[i][current_tag] = best_pr

    # Display top 5 probabilities for the current w
    sorted_current_probs = sorted(viterbi_table[i].
    print(f" Top 5 tag probabilities for '{current
    for tag, prob_log in sorted_current_probs[:5]:
        if math.isinf(prob_log): # Handle -inf for
            print(f" Tag: {tag}, Log Probability
        else:
            print(f" Tag: {tag}, Log Probability

    # print("\nViterbi Table after Recursion:", viterbi
    # print("\nBackpointer Table after Recursion:", bac

```

```

Tag: RB, Log Probability: -107.9066, Actual Probability: 0.00000000
Tag: VB, Log Probability: -108.2864, Actual Probability: 0.00000000
Tag: JJ, Log Probability: -110.1044, Actual Probability: 0.00000000

```

Processing word 13: 'the'

Top 5 tag probabilities for 'the':

```

Tag: DT, Log Probability: -102.4326, Actual Probability: 0.00000000
Tag: NN, Log Probability: -124.7422, Actual Probability: 0.00000000
Tag: JJ, Log Probability: -125.6793, Actual Probability: 0.00000000
Tag: NNS, Log Probability: -126.1994, Actual Probability: 0.00000000
Tag: PRP$, Log Probability: -127.8908, Actual Probability: 0.00000000

```

Processing word 14: 'mechanisms'

Top 5 tag probabilities for 'mechanisms':

```

Tag: NNS, Log Probability: -115.5574, Actual Probability: 0.00000000
Tag: NN, Log Probability: -126.1409, Actual Probability: 0.00000000
Tag: JJ, Log Probability: -127.0779, Actual Probability: 0.00000000
Tag: JJS, Log Probability: -129.4710, Actual Probability: 0.00000000
Tag: RB, Log Probability: -129.5364, Actual Probability: 0.00000000

```

Processing word 15: 'that'

Top 5 tag probabilities for 'that':

```

Tag: IN, Log Probability: -120.3917, Actual Probability: 0.00000000
Tag: WDT, Log Probability: -120.8970, Actual Probability: 0.00000000
Tag: DT, Log Probability: -122.8967, Actual Probability: 0.00000000
Tag: RB, Log Probability: -127.7844, Actual Probability: 0.00000000
Tag: VBN, Log Probability: -130.0090, Actual Probability: 0.00000000

```

Processing word 16: 'modi'

Top 5 tag probabilities for 'modi':

```

Tag: NN, Log Probability: -123.7988, Actual Probability: 0.00000000
Tag: JJ, Log Probability: -125.1979, Actual Probability: 0.00000000
Tag: VBP, Log Probability: -125.8832, Actual Probability: 0.00000000
Tag: VBZ, Log Probability: -125.9305, Actual Probability: 0.00000000
Tag: NNS, Log Probability: -126.0206, Actual Probability: 0.00000000

```

Processing word 17: 'has'

Top 5 tag probabilities for 'has':

```

Tag: VBZ, Log Probability: -128.2690, Actual Probability: 0.00000000
Tag: NN, Log Probability: -147.9336, Actual Probability: 0.00000000
Tag: IN, Log Probability: -149.2781, Actual Probability: 0.00000000
Tag: VBD, Log Probability: -149.7204, Actual Probability: 0.00000000
Tag: NNS, Log Probability: -149.7288, Actual Probability: 0.00000000

```

Processing word 18: 'created'

Top 5 tag probabilities for 'created':

```

Tag: VBN, Log Probability: -135.0019, Actual Probability: 0.00000000
Tag: VBD, Log Probability: -138.5531, Actual Probability: 0.00000000
Tag: JJ, Log Probability: -141.2906, Actual Probability: 0.00000000
Tag: VB, Log Probability: -141.8348, Actual Probability: 0.00000000
Tag: NN, Log Probability: -153.2489, Actual Probability: 0.00000000

```

```
print(f"\n--- Viterbi Termination and Backtracking
```

```
# Step 3: Termination - Find the path with the high
```



```

last_word_index = len(words) - 1

final_max_prob = -math.inf
last_best_tag = None

for tag, prob_log in viterbi_table[last_word_index]:
    if prob_log > final_max_prob:
        final_max_prob = prob_log
        last_best_tag = tag

print(f"\nFinal max log probability for the last wo
print(f"Corresponding best tag: {last_best_tag}")

# Step 4: Backtracking - Reconstruct the best tag s
predicted_tags = [None] * len(words)

# Start from the last best tag
current_tag = last_best_tag
predicted_tags[last_word_index] = current_tag

# Trace back through the backpointer table
for i in range(len(words) - 1, 0, -1):
    current_tag = backpointer_table[i][current_tag]
    predicted_tags[i-1] = current_tag

# Display the final decoded sequence
print("\n--- Decoded POS Tag Sequence (Viterbi) ---")
for i in range(len(words)):
    print(f"Word: '{words[i]}', Predicted Tag: {pre

print("\n--- Comparison with Original POS Tags ---")
original_tags_only = [tag for word, tag in selected
print(f"Original Tags: {original_tags_only}")
print(f"Predicted Tags: {predicted_tags}")

# Calculate accuracy
correct_predictions = sum(1 for pred_tag, orig_tag
accuracy = correct_predictions / len(words)
print(f"Accuracy for this tweet: {accuracy:.2%}")

```

--- Viterbi Termination and Backtracking for tweet: 'fact even the half baked mi

Final max log probability for the last word ('created'): -135.0019
 Corresponding best tag: VBN

--- Decoded POS Tag Sequence (Viterbi) ---

Word: 'fact', Predicted Tag: NN
 Word: 'even', Predicted Tag: RB
 Word: 'the', Predicted Tag: DT
 Word: 'half', Predicted Tag: NN

```
Word: 'baked', Predicted Tag: VBD
Word: 'mig', Predicted Tag: NN
Word: 'scheme', Predicted Tag: NN
Word: 'rahul', Predicted Tag: NN
Word: 'clearly', Predicted Tag: RB
Word: 'depending', Predicted Tag: VBG
Word: 'exploiting', Predicted Tag: VBG
Word: 'all', Predicted Tag: PDT
Word: 'the', Predicted Tag: DT
Word: 'mechanisms', Predicted Tag: NNS
Word: 'that', Predicted Tag: IN
Word: 'modi', Predicted Tag: NN
Word: 'has', Predicted Tag: VBZ
Word: 'created', Predicted Tag: VBN
```

--- Comparison with Original POS Tags ---

```
Original Tags: ['NN', 'RB', 'DT', 'NN', 'VBD', 'JJ', 'NN', 'VBD', 'RB', 'VBG',
Predicted Tags: ['NN', 'RB', 'DT', 'NN', 'VBD', 'NN', 'NN', 'NN', 'RB', 'VBG',
Accuracy for this tweet: 83.33%
```